

Ontrack Task5.1 Start Activity

SQL injection is a type of security vulnerability that occurs when an attacker inserts malicious SQL code into a query, which is then executed by a database server. This can lead to unauthorized access, data theft, data manipulation, and even complete system compromise. SQL injection attacks are mostly directed towards web applications that use SQL databases in the backend.

Run a simple SQL based command line program

1. First let's start the mysql database on the kali Linux, to do this use the command

sudo systemctl start mysql

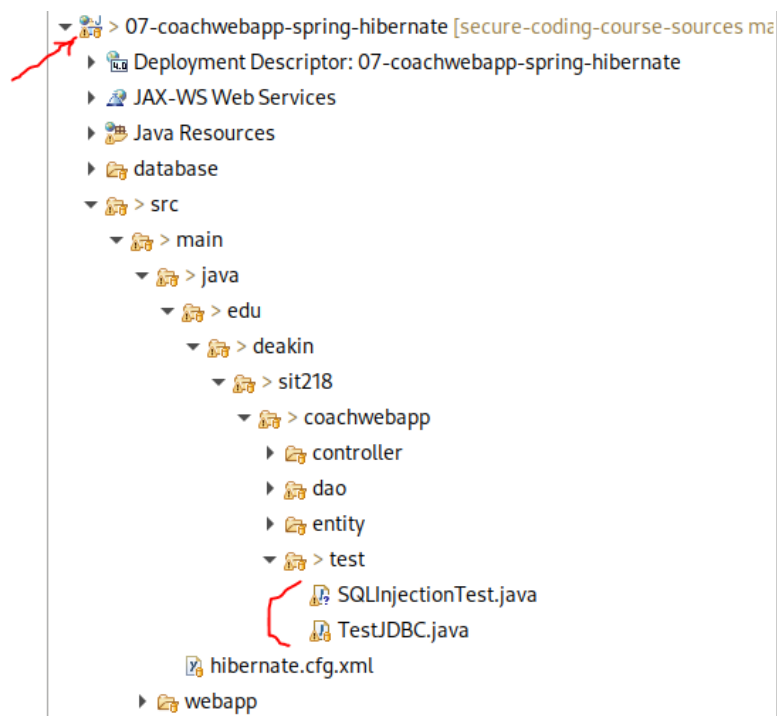
2. Start eclipse

cd /opt/eclipse/eclipse

sudo ./eclipse

when prompted for password enter 'kali'

3. Navigate to the 07-coachwebapp-spring-hibernate project in the project explorer.



4. Expand the 'src' folder until you see the SQLInjectionTest.java file. Double click the file and you will see the code loaded in the eclipse workspace.

```

1 package edu.deakin.sit218.coachwebapp.test;
2
3 import java.util.Scanner;
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 public class SQLInjectionTest {
11
12
13     public static void main(String[] args) {
14         //Ask user to give his/her name
15         Scanner scanner = new Scanner(System.in);
16         System.out.print("Enter your name: ");
17         String name = scanner.nextLine();
18         scanner.close();
19
20         String jdbcURL = "jdbc:mysql://localhost/coachappdb?" +
21             "user=coachdbadmin&password=coachdbadmin";
22         try {
23             System.out.println("Connecting to Database");
24             Connection conn =
25                 DriverManager.getConnection(jdbcURL);
26
27             //Perform a query to select all users whose name
28             //is equal to the name entered in the command line\
29             /*
30             Statement statement = conn.createStatement();
31             String query = "SELECT * FROM client WHERE name = '"+name+"'";
32             System.out.print("Executing query: ");
33             System.out.println(query);
34             ResultSet result = statement.executeQuery(query);
35             */
36

```

Make the following changes before proceeding.

Uncomment the code between lines 29 and 35 by deleting the characters `/*` and `*/` as shown below:

```

27         //Perform a query to select all users whose name
28         //is equal to the name entered in the command line\
29         /*
30         Statement statement = conn.createStatement();
31         String query = "SELECT * FROM client WHERE name = '"+name+"'";
32         System.out.print("Executing query: ");
33         System.out.println(query);
34         ResultSet result = statement.executeQuery(query);
35         */
36

```

Change to

```

29         Statement statement = conn.createStatement();
30         String query = "SELECT * FROM client WHERE name = '"+name+"'";
31         System.out.print("Executing query: ");
32         System.out.println(query);
33         ResultSet result = statement.executeQuery(query);
34

```

Also comment the lines between 38 and 41 as shown below by adding `/*` and `*/` :

```

37
38         PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
39         statement.setString(1, name);
40         System.out.print(statement.toString() + "\n");
41         /*
42         ResultSet result = statement.executeQuery();
43         */

```

Change to

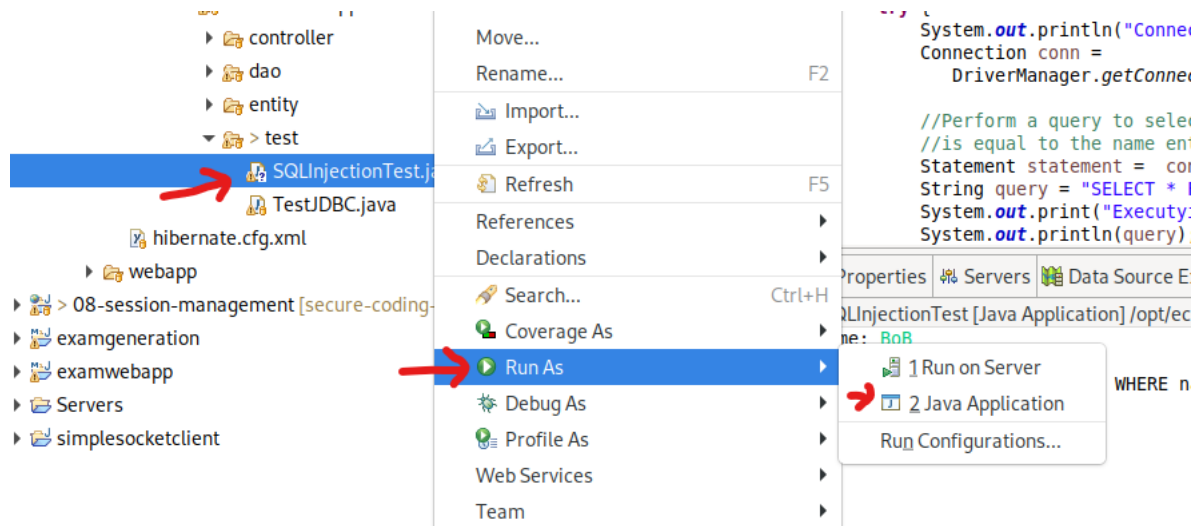
```

35
36
37
38     PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
39     statement.setString(1, name);
40     System.out.print(statement.toString() + "\n");
41     ResultSet result = statement.executeQuery();
42 */
43

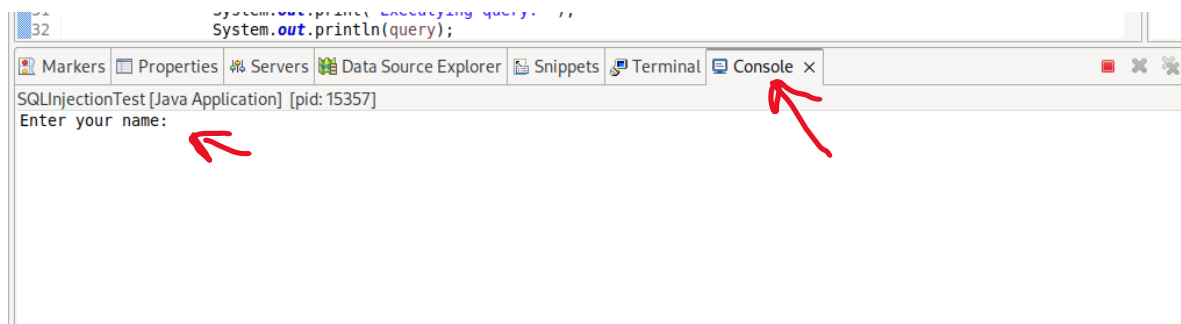
```

Now save the code by pressing ctrl+s.

5. Right click on the SQLInjectionTest.java file under test folder in the project explorer and select **Run As** and select **Java Application**.



You will notice the code running in the console at the bottom of the eclipse window.



Activity-1: Test the application to retrieve user information

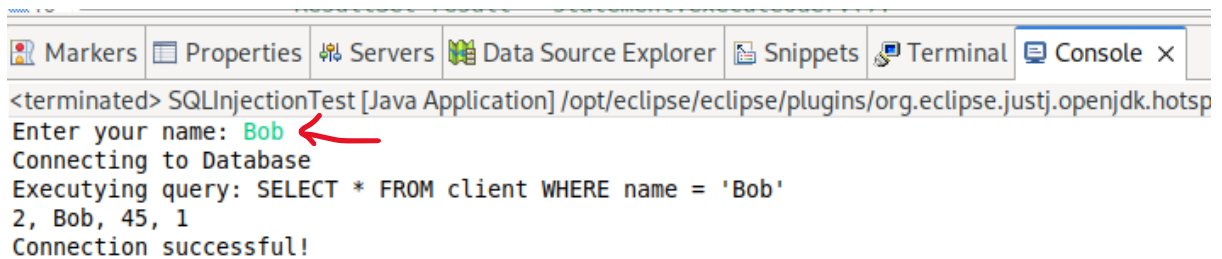
The aim is to first retrieve some existing information from the database table. The piece of code that uses the input given by the user to search the database is shown below.

```

Statement statement = conn.createStatement();
String query = "SELECT * FROM client WHERE name = '"+name+"'";
System.out.print("Executing query: ");
System.out.println(query);
ResultSet result = statement.executeQuery(query);

```

If we want to fetch the information regarding client 'Bob' we can enter this in the console and the application will retrieve the matching the results. Now enter the value 'Bob' in the console where it prompts "Enter your name:"



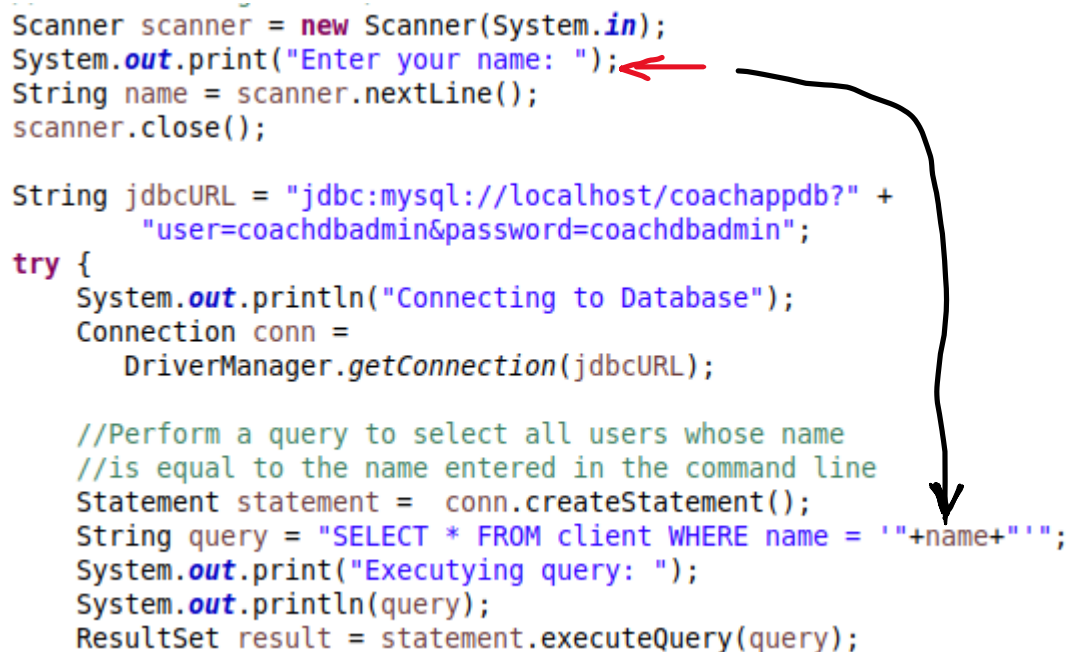
The screenshot shows the Eclipse IDE's console window. The title bar indicates the application is 'SQLInjectionTest [Java Application]'. The console output shows the following sequence of events: a prompt 'Enter your name: Bob' with a red arrow pointing to the input 'Bob'; a message 'Connecting to Database'; an SQL query 'SELECT * FROM client WHERE name = 'Bob'' being executed; the results '2, Bob, 45, 1' displayed; and a final message 'Connection successful!'.

```
<terminated> SQLInjectionTest [Java Application] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotsp
Enter your name: Bob
Connecting to Database
Executing query: SELECT * FROM client WHERE name = 'Bob'
2, Bob, 45, 1
Connection successful!
```

You can see the details of the user Bob displayed which contains the ID, Name, Age and Number of workouts.

Activity-2: Perform SQL injection

To perform SQLInjection the idea is to enter an input that allows us to fetch all the information which we are not authorised or do not have access to. Here we do not know the other users in the database. But we can trick the application to fetch all the user's information due to the vulnerable code. The vulnerability in the code is it fetches the users input and without any validations passes it to the SQL query. So there SQL injections are a result of lack of input validations and secondly generating SQL queries using the user input. Let's see this in action.



The code snippet shows a Java program that takes user input and uses it in an SQL query. A red arrow points to the input line, and a black arrow points from the input variable to the query string, illustrating how the input is directly passed to the SQL query without validation.

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = scanner.nextLine();
scanner.close();

String jdbcURL = "jdbc:mysql://localhost/coachappdb?" +
    "user=coachdbadmin&password=coachdbadmin";

try {
    System.out.println("Connecting to Database");
    Connection conn =
        DriverManager.getConnection(jdbcURL);

    //Perform a query to select all users whose name
    //is equal to the name entered in the command line
    Statement statement = conn.createStatement();
    String query = "SELECT * FROM client WHERE name = '"+name+"'";
    System.out.print("Executing query: ");
    System.out.println(query);
    ResultSet result = statement.executeQuery(query);
```

As you can see the input is passed to the SQL query without any validations. This makes the code vulnerable to SQL injection. When we enter the input as eg. **Bob' OR 1=1--'** then this will translate into the code as : here input is: **Bob' OR 1=1--'**

SELECT * FROM client WHERE name = 'Bob' OR 1=1--'

The value '1=1' is also called as tautology and is always TRUE which will fetch all the values. The '--' in the injected query acts as comment and everything after that will be disregarded by the SQL query. And when this is passed to the injected OR parameter the SQL query will fetch either Bob OR everything. Lets try this in the program.

Run the code again as done before using the Run As and enter the below input

Bob' OR 1=1 --'

```
Enter your name: Bob' OR 1=1 --'
Connecting to Database
Executying query: SELECT * FROM client WHERE name = 'Bob' OR 1=1 --'
1, Test, 34, 0
2, Bob, 45, 1
3, Tom, 24, 0
4, Paul, 56, 1
5, Test, 44, 1
6, Rolando, 20, 0
7, Rolando, 20, 0
Connection successful!
```

You have now done the SQL injection attack and retrieved all the values in the table.

Activity-3: Preventing SQL injection

The way to make the SQL query safer is to validate the input and don't permit unintended input and also use preparedstaments to prepare the query and set the correct user values in placeholders instead of generating the query with the user's input. In the code file **SQLInjectionTest.java** make the following changes:

We need to comment the vulnerable code and enable the safe SQL query statement. This can done by commenting lines 30-34, Add `/*` before the Statement statement = conn.createStatement(); line 30 and after the ResultSet line 34 add `*/`. Also uncomment the lines from 36 to 42.

Before changes in the code:

```
29         Statement statement = conn.createStatement();
30         String query = "SELECT * FROM client WHERE name = '"+name+"'";
31         System.out.print("Executying query: ");
32         System.out.println(query);
33         ResultSet result = statement.executeQuery(query);
34
35         /*
36
37         PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
38         statement.setString(1, name);
39         System.out.print(statement.toString() + "\n");
40         ResultSet result = statement.executeQuery();
41
42         */
```

After changes in the code.

```

26
27 //Perform a query to select all users whose name
28 //is equal to the name entered in the command line\
29 /*
30 Statement statement = conn.createStatement();
31 String query = "SELECT * FROM client WHERE name = '"+name+"'";
32 System.out.print("Executing query: ");
33 System.out.println(query);
34 ResultSet result = statement.executeQuery(query);
35 */
36
37
38 PreparedStatement statement = conn.prepareStatement("select * from client where name = ?");
39 statement.setString(1, name);
40 System.out.print(statement.toString() + "\n");
41 ResultSet result = statement.executeQuery();
42
43

```

Now run the java code as done previously. And try to enter the malicious input when it prompts for name.

You can now see the connection is successful but nothing is retrieved from the database.

```

<terminated> SQLInjectionTest [Java Application] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x
Enter your name: Bob' OR 1=1 --'
Connecting to Database
com.mysql.cj.jdbc.ClientPreparedStatement: select * from client where name = 'Bob' OR 1=1 --''
Connection successful!

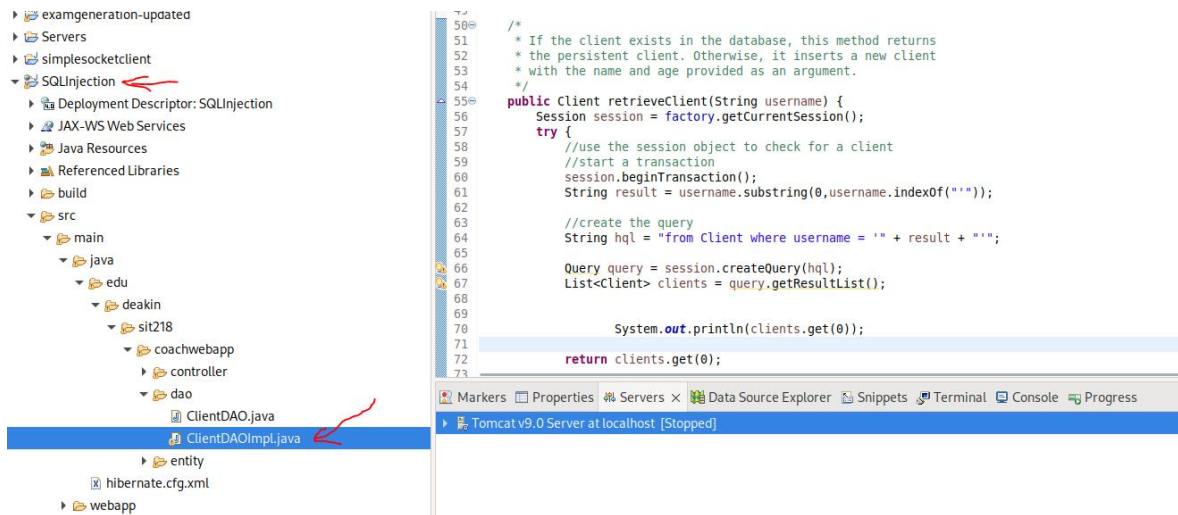
```

Reason for this is that you first prepare the statement and then insert the users value in to the query rather than generating the query after the user enters the input. This protects the query from being misused.

Activity-4: SQL injection test in the webapp.

Vulnerable SQL queries are used in user authentication then this can lead to users getting unauthorised access.

In the Project explorer, open the **SQLInjection webapp**. Expand the src folder and double click on ClientDAOImpl.java



Few corrections in the code to run the web app correctly

In the code below, comment (using '//') the line number 61 and change the variable name in line 64 from 'result' to 'username'.

```

55 public Client retrieveClient(String username) {
56     Session session = factory.getCurrentSession();
57     try {
58         //use the session object to check for a client
59         //start a transaction
60         session.beginTransaction();
61         String result = username.substring(0,username.indexOf(""));
62
63         //create the query
64         String hql = "from Client where username = ' " + result + "'";
65
66         Query query = session.createQuery(hql);
67         List<Client> clients = query.getResultList();
68
69         System.out.println(clients.get(0));
70
71         return clients.get(0);
72     }
73 }

```

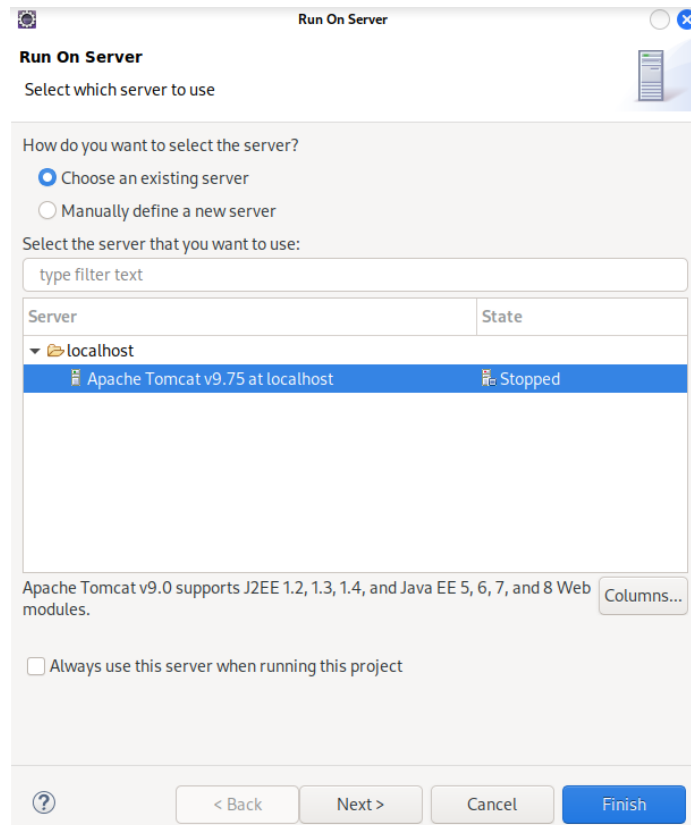
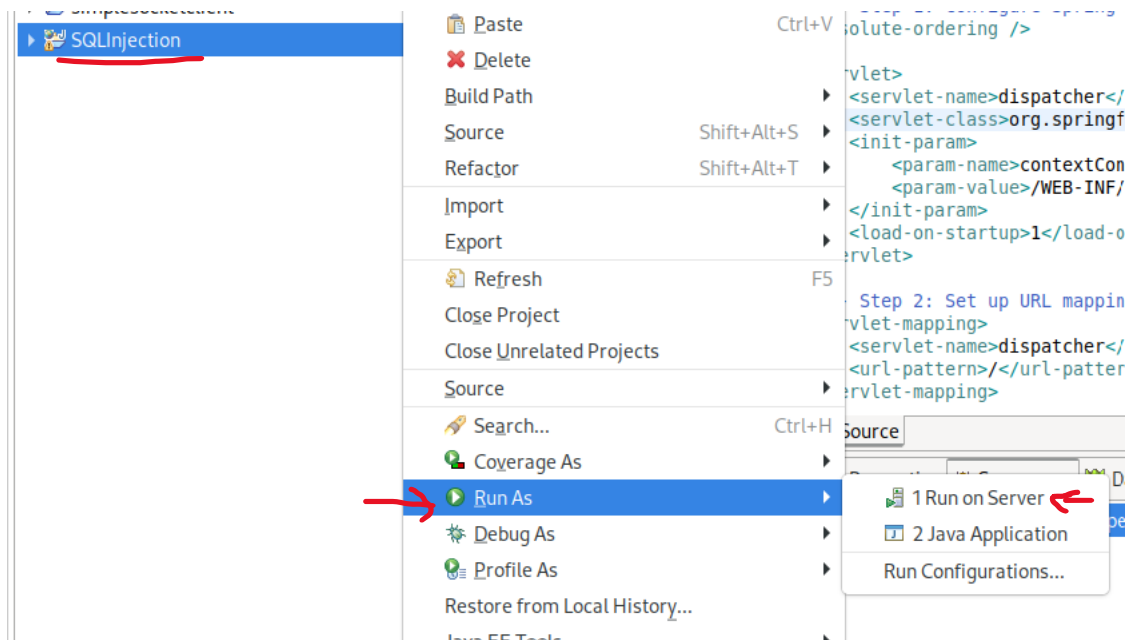
After changes in the code:

```

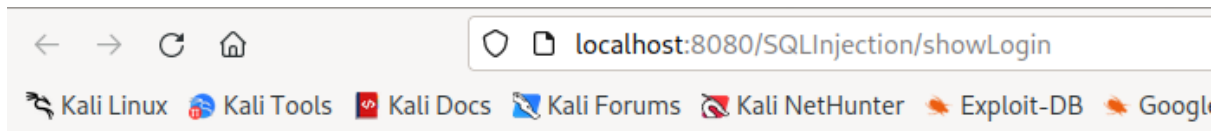
60     session.beginTransaction();
61     // String result = username.substring(0,username.indexOf(""));
62
63     //create the query
64     String hql = "from Client where username = ' " + username + "'";
65
66     Query query = session.createQuery(hql);
67     List<Client> clients = query.getResultList();
68
69     System.out.println(clients.get(0));
70
71     return clients.get(0);
72 }

```

Save the file and right click on the SQLInjection project and Run AS on Server.



You will see the app loaded in the webbrowser



Please Login

User name:

Password:

Login

Try to login to the application without the password. One of the usernames configured in this application is 'Alice'. Login using this username.

Continue to the Task 5.1P.